

地址翻译硬件与 OS 虚拟内存

操作系统怎样把地址空间策略交给硬件，并在翻译不能继续时收回控制权

408 · Cross-Subject Bridge X-B02

母接口：每条边只承载一种语义

OS Mapping Decision

→ encode →

PTE/Page-Table State

→ consume →

MMU/TLB

硬件消费映射状态后只产生两类接口结果：成功时 $\xrightarrow{\text{translate + check}}$ 合法的 *PA + permission*；失败时 $\xrightarrow{\text{fault handoff}}$ 把原因与现场交回 OS。若 OS 能修复，则通过“更新映射并处理旧翻译”建立新的可消费状态，原访问才在明确的 *retry* 点重新尝试。

这里不再用一条裸箭头同时表示因果、数据流和控制权交接：**encode / consume / translate+check / fault handoff / retry** 是不同接口动作。

先定义接口两侧的词

- **MMU** (Memory Management Unit) 是 CPU 内负责把 VA 按页表翻译为 PA、同时检查权限的硬件单元。
- **页表** 是 OS 建立、硬件读取的映射数据结构；**PTE** 是其中描述一个虚拟页的单个表项。
- **TLB** 是 MMU 使用的翻译缓存，只保存近期翻译结果；TLB 没命中时可继续 *page walk*，不能直接判为缺页。
- **Page Fault** 是当前访问不能按现有映射或权限完成时产生的可观察 *fault*；OS 可能装入页面、修改权限后重试，也可能拒绝访问。
- **Retry** 表示修复后重新执行导致 *fault* 的那次访问；它不是“随便再访问一次”，而是要保持原指令语义和保存现场的一致性。

左侧 CO07 提供每次访问的硬件消费规则，右侧 OS-04 提供地址空间、驻留和 *fault* 修复策略；Bridge 只说明映射状态如何跨越两侧。

定位、Owner 与停止边界

- **左侧 Owner**：CO07 地址翻译硬件，负责 TLB、*page walk*、PTE 可见字段、VA 到 PA 的翻译与访问权限检查。
- **右侧 Owner**：OS-04 虚拟内存与页生命周期，负责地址空间、页表构造、页分配/驻留、缺页处理、COW 与置换策略。
- **本 Bridge Owns**：映射状态如何成为硬件输入；翻译结果、权限和 *fault* 如何成为 OS 的输入；修复后为何可以重试。
- **Stops**：不重新讲 TLB 内部替换、Cache 数据副本、文件页缓存或具体 ISA 的全部实现；它们分别回到 CO07、CO06、OS 文件/存储 Owner。

1 先区分对象：地址、映射状态与结果

一次访问的输入不能只写成一个 VA，至少应保留

$$R = (VA, access\ type, privilege, size).$$

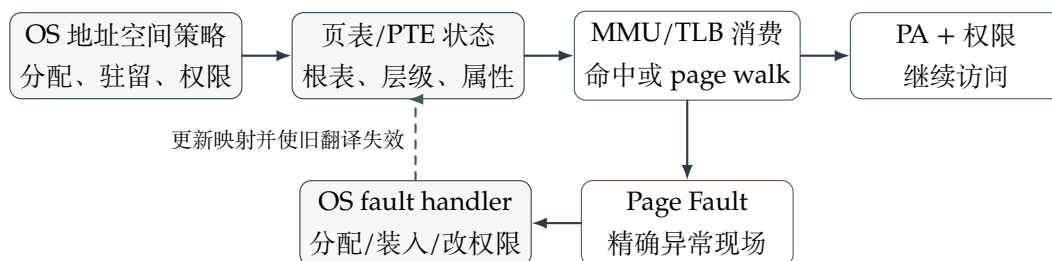
OS 维护的不是“所有 VA 的答案表”，而是稀疏的层级映射状态。Linux 文档明确把页表描述为“把 CPU 看到的虚拟地址映射到外部内存总线看到的物理地址”的层级结构；层级结构还能用高层条目表达大范围映射，避免为大空洞分配大量低层条目¹。

硬件消费映射后，接口输出应写成带状态的结果包：

$$T = \{PA, permission, memory\ attributes, status\}.$$

当 $status = success$ 时，PA 才能交给后续 Cache/内存路径；当 $status = fault$ 时，输出的是“不能继续的原因与现场”，不是一个可被 Cache 试探的合法 PA。

2 母模型：配置、消费、失败、修复



2.1 为什么必须是这个顺序

“OS 先决定、硬件再消费”不是教学口号，而是责任边界：OS 拥有全局资源与策略，硬件拥有每次访问必须快速、可强制执行的局部检查。RISC-V 的 `satp` 保存根页表物理页号、ASID 与地址翻译模式；它把地址空间选择交给监督态软件配置，而不是让每次用户访问调用 OS²。

2.2 一个最小例子：首次访问尚未驻留的页面

设进程访问 $VA = 0x4000$ ，OS 已建立该区域“可读”的虚拟地址范围，但物理页尚未驻留。硬件沿当前页表查找时发现映射状态不能完成这次 load，于是产生 load page fault。OS 读取 fault 原因与出错地址，分配或装入物理页，写入合法 PTE，按平台要求处理旧翻译，再从保存的指令位置恢复。恢复后同一条 load 再次经过翻译；这一次成功才进入 Cache/内存访问。

这里的“retry”不是重复计算的装饰：如果 faulting instruction 已经产生架构可观察的保存位置，修复后的映射必须让它重新获得与原语义一致的执行机会；若访问违法或无法修复，OS 应转入信号/终止分支，而不是伪装成功结果。

3 边界表：三类“没有找到”不能合并

¹Linux Kernel Documentation, Page Tables.

²RISC-V ISA Manual, Supervisor Address Translation and Protection (satp).

对象 A	≠ 对象 B	真正区别与题目信号	混淆后果
TLB miss	≠ Page Fault	前者缺的是翻译缓存副本，page walk 仍可能成功；后者表示当前访问不能按现有映射或权限继续，需要 fault 处理。	把一次可由硬件继续完成的翻译 miss 错算成 OS 异常甚至磁盘 I/O。
Page Fault	≠ Cache miss	Page Fault 发生在合法物理身份尚不能生成或权限不允许的翻译阶段；Cache miss 发生在已有合法访问目标后的数据副本层。	在尚未得到合法 PA 时继续计算 Cache，或把数据缓存 miss 当成页不驻留。
Hardware Cache	≠ OS Page Cache	前者属于硬件数据访问层级；后者由 OS 按文件对象/页索引管理文件内容副本。	把硬件 Cache fill/replace 与文件页驻留、writeback 交给错误的 Owner。

因此一次访问可以先 TLB miss、page walk 成功，再发生 Cache miss；也可以在映射或权限检查处直接 fault。“没有找到”必须先说明缺失对象和 Owner。

4 架构证据：以 RISC-V 为锚点，不冒充所有 ISA 相同

RISC-V 监督态规范给出了适合本 Bridge 的最小可观察协议：trap 进入 S-mode 时，sepc 保存被中断或发生异常的指令地址，scause 保存事件代码；原因表区分 U-mode/S-mode environment call、load/store page fault 等事件。SRET 根据 SPP 恢复到陷阱前的特权级。于是共同接口可以拆成四个有主语的动作：硬件保存精确现场；内核读取并分类原因；内核修复或拒绝访问；体系结构返回路径恢复到定义明确的执行位置或终止分支³。

这只是一个架构实例。x86、ARM 和 RISC-V 在寄存器名称、页表格式、硬件/软件 page walk 与返回指令上不同；408 层面只保留“保存现场、识别原因、修复或拒绝、恢复或终止”这组接口职责。遇到具体 ISA 题目，必须回到该 ISA Owner 的入口协议。

5 最小调用接口

当一道题同时需要 OS 的 mapping/fault 语义和 CO07 的 TLB/MMU 翻译行为时调用本 Bridge。第一观察点是：OS 预先提供了什么映射状态，硬件当前正在消费哪一份状态，成功或 fault 的交接点在哪里？

完整的跨科落笔顺序、TLB miss / Page Fault / Cache miss 分流、旧翻译检查与 retry 训练统一进入同目录 地址翻译软硬件交接.md；若题目只问单侧内部机制，则立即回到 CO07 或 OS-04，不在 Bridge 中重复展开。

压缩复原题

页表根地址改变后，为什么不能只说“下一次访问自动正确”？答案必须补出：地址空间状态已改变；旧 TLB 副本可能仍对应旧映射；系统需要按具体 ISA/平台规则完成翻译失效或隔离；随后访问才重新消费新 PTE。这里考的是 **状态交接与合法性不变量**，不是背某个寄存器名字。

³RISC-V ISA Manual, Supervisor trap and address-translation registers.

6 Bridge 交接不变量

1. **身份不变量**：只有合法翻译得到的 PA 才能进入后续数据访问路径。
2. **权限不变量**：命中 TLB 或 Cache 不能绕过本次访问的权限检查。
3. **状态一致性**：OS 改变映射后，任何仍可消费的旧翻译都必须被失效、隔离或用 ASID 等机制区分。
4. **精确恢复**：可修复 fault 应回到定义明确的指令/访问点；不可修复 fault 不能伪造数据返回。
5. **边界可辨识**：TLB miss、page fault、Cache miss 的处理者和重试点必须不同。

一句话带走

OS 把“允许哪些虚拟地址如何存在”编码为页表状态，MMU/TLB 把它消费成一次访问的物理身份与权限结果；当结果不能合法生成时，fault 把控制权交回 OS，OS 修复或拒绝后，访问才可能在明确的 retry 点继续。